

Details:

Name: Siddhesh Abasaheb Mane

PRN: 202401040187

Batch: A2

Roll No.: 46

Title

Personalized Course Recommendation System Using Prolog

Aim

To design and implement a rule-based Course Recommendation System using Prolog that recommends suitable courses to students based on their skills, interests, and course prerequisites.

Experiment Objectives

- To understand the concepts of logic programming using Prolog.
- To represent student information using facts and predicates.
- To define rules for checking course eligibility.
- To recommend courses according to a student's interests.
- To demonstrate variables, rules, unification, logical AND, and backtracking in Prolog.
- To implement a simple knowledge-based expert system for personalized course recommendation.

Experiment Outcome

- To Represent knowledge using Prolog facts and rules.
- Query a Prolog knowledge base using variables.
- To Perform logical inference using predefined rules.
- To Check whether a student satisfies the prerequisite of a course.

- Generate personalized course recommendations.
- Understand the role of Prolog in developing rule-based AI systems.

Theory

1. Introduction to Prolog

Prolog stands for PROgramming in LOGic. It is a logic programming language widely used for artificial intelligence and knowledge-based systems.

Unlike conventional programming languages, where the programmer specifies a sequence of instructions, Prolog represents knowledge using facts and rules. The user then provides queries, and Prolog uses logical inference, unification, and backtracking to find solutions.

A Prolog program mainly consists of:

- Facts
- Rules
- Queries

For example:

```
skill(siddhesh,java).
```

is a fact stating that Siddhesh has Java as a skill.

A rule such as:

```
has_prerequisite(Student,Course) :- prerequisite(Course,Skill), skill(Student,Skill).
```

states that a student has a prerequisite for a course if the required skill of that course is present in the student's skills.

2. Knowledge Representation

The system represents information about:

- Students
- Skills
- Interests
- Course categories
- Prerequisites

For example:

```
skill(siddhesh,java). interest(siddhesh,backend). course(spring_boot,backend).  
prerequisite(spring_boot,java).
```

From these facts, Prolog can infer that Spring Boot is a suitable course for Siddhesh because he is interested in backend development and already knows Java.

3. Eligibility Checking

The system uses the `eligible/2` predicate to determine whether a student can take a course.

A course can be considered eligible when:

- It has no prerequisite, or
- The student possesses the required prerequisite skill.

4. Course Recommendation

The `recommend/2` rule combines:

- Student interest
- Course category
- Student eligibility

The recommendation rule is:

```
recommend(Student,Course) :- interest(Student,Category), course(Course,Category),  
eligible(Student,Course).
```

This means:

A course is recommended to a student if the student's interest matches the course category and the student is eligible for that course.

Algorithm

1. Start the program.
2. Store student information in the knowledge base.
3. Store the skills possessed by each student.
4. Store the interests of each student.
5. Store available courses and their categories.
6. Store prerequisite skills for courses.
7. Check whether a course has no prerequisite.
8. If a course has a prerequisite, check whether the student possesses the required skill.
9. Match the student's interest with the course category.
10. If both interest and eligibility conditions are satisfied, recommend the course.
11. Display the recommended courses.
12. Stop.

Code

```
student(siddhesh).
student(rahul).
student(priya).
student(amit).
skill(siddhesh,java).
skill(siddhesh,sql).
skill(siddhesh,html).
skill(rahul,python).
skill(rahul,statistics).
skill(ram,html).
skill(ram,css).
skill(ram,javascript).
skill(amit,cpp).
skill(amit,dsa).
interest(siddhesh,backend).
interest(siddhesh,ai).
interest(rahul,data_science).
interest(ram,frontend).
interest(amit,competitive_programming).
course(spring_boot,backend).
course(rest_api,backend).
course(database_systems,backend).
course(machine_learning,ai).
course(deep_learning,ai).
course(natural_language_processing,ai).
course(python_data_science,data_science).
course(statistics_for_data_science,data_science).
course(react,frontend).
course(javascript,frontend).
```

```
course(ui_ux,frontend).
course(dsa,competitive_programming).
course(advanced_cpp,competitive_programming).
prerequisite(spring_boot,java).
prerequisite(rest_api,java).
prerequisite(database_systems,sql).
prerequisite(machine_learning,python).
prerequisite(python_data_science,statistics).
prerequisite(react,javascript).
prerequisite(advanced_cpp,cpp).
prerequisite(dsa,cpp).
no_prerequisite(javascript).
no_prerequisite(ui_ux).
no_prerequisite(database_systems).
has_prerequisite(Student,Course) :- prerequisite(Course,Skill), skill(Student,Skill).
eligible(_Course) :- no_prerequisite(Course).
eligible(Student,Course) :- has_prerequisite(Student,Course).
recommend(Student,Course) :- interest(Student,Category), course(Course,Category),
eligible(Student,Course).
```

SWISH

[click](#)

Queries and Results

```

course(machine_learning,X).
X = ai
prerequisite(spring_boot,X).
X = java
has_prerequisite(siddhesh,spring_boot).
true
has_prerequisite(siddhesh,machine_learning).
false
eligible(siddhesh,spring_boot).
true
eligible(siddhesh,javascript).
true
false
eligible(siddhesh,advanced_cpp).
false
eligible(amit,C).
C = javascript
C = ui_ux
C = database_systems
C = advanced_cpp
C = dsa

```

```

student(siddhesh).
true
student(rohan).
false
skill(siddhesh,X).
X = java
X = sql
X = html
skill(X,css).
X = ram
interest(amit,X).
X = competitive_programming
interest(X,ai).
X = siddhesh
course(X,backend).
X = spring_boot
X = rest_api
X = database_systems
course(machine_learning,X).
X = ai

```



```
recommend(siddhesh,C).  
C = spring_boot  
C = rest_api  
C = database_systems  
C = database_systems  
  
recommend(rahul,C).  
C = python_data_science
```

Result Analysis

The Course Recommendation System successfully represents student profiles, skills, interests, courses, and prerequisites using Prolog facts.

The system uses logical rules to determine course eligibility and generate recommendations. For example, Siddhesh has Java and SQL skills and is interested in backend development and AI. The system can therefore recommend suitable backend courses such as Spring Boot, REST API, and Database Systems when their eligibility conditions are satisfied.

Similarly, Rahul is interested in Data Science and possesses Python and Statistics skills, allowing the system to recommend relevant Data Science courses.

Conclusion

The AI-Based Personalized Course Recommendation System was successfully implemented using Prolog. The system uses facts and logical rules to represent student information, course information, interests, skills, and prerequisites.

The experiment demonstrates important Prolog concepts such as facts, predicates, variables, rules, logical AND. Based on the available knowledge, the system can determine student eligibility and provide personalized course recommendations.

Thus, Prolog can be effectively used to develop simple rule-based expert systems and intelligent recommendation systems.

Github Link

[click](#)

Colab Link

[click](#)